

ASSIGNMENT: 6

1. Write a Program to perform the following operations on the array:

- a. Read Array**
- b. Print Array**
- c. Find Minimum Number from Array**
- d. Find Maximum Number from Array**

Algorithm:

1. Start
2. Declare array and variables (n, min, max, i)
3. Input array size n
4. Loop from i = 0 to n-1:
 - Read a[i]
5. Print array elements using a loop
6. Initialize min = max = a[0]
7. Loop from i = 1 to n-1:
 - If a[i] < min, update min
 - If a[i] > max, update max
8. Print min and max
9. End

Code:

```
#include <stdio.h>
```

```
int main() {  
    int a[100], n, i, min, max;  
  
    printf("Enter size of array: ");  
    scanf("%d", &n);  
  
    printf("Enter elements:\n");  
    for(i = 0; i < n; i++)  
        scanf("%d", &a[i]);  
}
```

```
printf("Array elements: ");  
for(i = 0; i < n; i++)  
    printf("%d ", a[i]);  
  
min = max = a[0];  
for(i = 1; i < n; i++) {  
    if(a[i] < min) min = a[i];  
    if(a[i] > max) max = a[i];  
}  
  
printf("\nMinimum = %d", min);  
printf("\nMaximum = %d", max);  
  
return 0;  
}
```

- 2. Write a program to perform the following operations on the array:**
- a. Read Array**
 - b. Print Array**
 - c. Search an Element from given array using linear search. Print the location of the element.**

Algorithm:

1. Start
2. Declare array and variables (n, key, found, i)
3. Input size n
4. Loop i = 0 to n-1:
 - Read a[i]
5. Print array
6. Input the element key to search
7. Loop i = 0 to n-1:
 - If a[i] == key, set found = 1 and print position
 - Break loop
8. If found == 0, print "Not found"
9. End

Code:

```
#include <stdio.h>
```

```
int main() {  
    int a[100], n, i, key, found = 0;  
  
    printf("Enter size of array: ");  
    scanf("%d", &n);  
  
    printf("Enter elements:\n");  
    for(i = 0; i < n; i++)  
        scanf("%d", &a[i]);  
  
    printf("Array elements: ");  
    for(i = 0; i < n; i++)  
        printf("%d ", a[i]);  
}
```

```
printf("\nEnter element to search: ");
scanf("%d", &key);

for(i = 0; i < n; i++) {
    if(a[i] == key) {
        printf("Element found at position %d\n", i + 1);
        found = 1;
        break;
    }
}

if(!found)
    printf("Element not found.\n");

return 0;
}
```

- 3. Write a program to perform the following operations on the array:**
- a. Read Array**
 - b. Print Array**
 - c. Print sorted array using Bubble sort.**

Algorithm:

1. Start
2. Declare array and variables (n, i, j, temp)
3. Input array size n and elements
4. Print original array
5. Loop i = 0 to n-2:
 - Loop j = 0 to n-i-2:
 - If $a[j] > a[j+1]$, swap them
6. Print sorted array
7. End

Code:

```
#include <stdio.h>
```

```
int main() {
    int a[100], n, i, j, temp;

    printf("Enter size of array: ");
    scanf("%d", &n);

    printf("Enter elements:\n");
    for(i = 0; i < n; i++)
        scanf("%d", &a[i]);

    printf("Original Array: ");
    for(i = 0; i < n; i++)
        printf("%d ", a[i]);

    // Bubble Sort
    for(i = 0; i < n-1; i++) {
        for(j = 0; j < n-i-1; j++) {
```

```
        if(a[j] > a[j+1]) {
            temp = a[j];
            a[j] = a[j+1];
            a[j+1] = temp;
        }
    }
}

printf("\nSorted Array: ");
for(i = 0; i < n; i++)
    printf("%d ", a[i]);

return 0;
}
```

- 4. Write a Program to Perform the following operations on a 2D Array:**
- a. Addition of two matrices.**
 - b. Transpose of a matrix.**
 - c. Display diagonal elements of the matrix**
 - d. Display upper diagonal & lower diagonal elements of the matrix**

Algorithm:

1. Start
2. Declare 2D arrays a, b, sum, trans and variables (i, j, n)
3. Input size n of square matrix
4. Input elements of matrix A and B
5. Addition:
 - Loop i = 0 to n-1, j = 0 to n-1:
 - $\text{sum}[i][j] = a[i][j] + b[i][j]$
6. Transpose:
 - Loop i = 0 to n-1, j = 0 to n-1:
 - $\text{trans}[j][i] = a[i][j]$
7. Diagonal Elements:
 - Loop i = 0 to n-1:
 - Print $a[i][i]$
8. Upper Diagonal:
 - Loop i = 0 to n-1, j = i+1 to n-1:
 - Print $a[i][i]$
9. Lower Diagonal:
 - Loop i = 1 to n-1, j = 0 to i-1:
 - Print $a[i][j]$
10. End

Code:

```
#include <stdio.h>
```

```
int main() {  
    int a[10][10], b[10][10], sum[10][10], trans[10][10];  
    int i, j, n;
```

```
printf("Enter size of square matrix (n x n): ");
scanf("%d", &n);
```

```
printf("Enter first matrix:\n");
for(i = 0; i < n; i++)
    for(j = 0; j < n; j++)
        scanf("%d", &a[i][j]);
```

```
printf("Enter second matrix:\n");
for(i = 0; i < n; i++)
    for(j = 0; j < n; j++)
        scanf("%d", &b[i][j]);
```

```
// Matrix Addition
printf("Addition of matrices:\n");
for(i = 0; i < n; i++) {
    for(j = 0; j < n; j++) {
        sum[i][j] = a[i][j] + b[i][j];
        printf("%d ", sum[i][j]);
    }
    printf("\n");
}
```

```
// Transpose
printf("Transpose of first matrix:\n");
for(i = 0; i < n; i++) {
    for(j = 0; j < n; j++) {
        trans[j][i] = a[i][j];
    }
}
for(i = 0; i < n; i++) {
    for(j = 0; j < n; j++)
        printf("%d ", trans[i][j]);
    printf("\n");
}
```

```
// Diagonal Elements
printf("Main Diagonal Elements: ");
for(i = 0; i < n; i++)
    printf("%d ", a[i][i]);
```



```

printf("\nUpper Diagonal: ");
for(i = 0; i < n; i++)
    for(j = i+1; j < n; j++)
        printf("%d ", a[i][j]);

printf("\nLower Diagonal: ");
for(i = 1; i < n; i++)
    for(j = 0; j < i; j++)
        printf("%d ", a[i][j]);

return 0;
}

```

- 5. Write a Program to Accept String from the user. Perform the following operations on the String (Without Using Built in Functions):**
- Print String Length**
 - Print String in reverse order**
 - Copy given string into other string and print both strings.**
 - Accept two strings from user and compare both. Print if they are equal. Or which one is greater or which one is small.**
 - Accept two strings from user and print its concatenated string.**

Algorithm:

- Start
- Declare strings: str1, str2, s1, s2, str3
- Input str1
- Length:
 - Loop through str1 until '\0', count characters
- Reverse:
 - Print characters from last to first index
- Copy:
 - Loop through str1, copy to str2
- Compare:
 - Input s1, s2.
 - Loop through both:
 - If characters differ, break and compare
- Concatenate:
 - Copy s1 to str3

- Append s2 at the end of str3
9. Print results
 - 10.End

Code:

```
#include <stdio.h>

int main() {
    char str1[100], str2[100], str3[100];
    int i, len = 0, cmp = 0;

    printf("Enter a string: ");
    scanf("%s", str1);

    // Length
    while(str1[len] != '\0')
        len++;

    printf("Length = %d\n", len);

    // Reverse
    printf("Reverse = ");
    for(i = len - 1; i >= 0; i--)
        printf("%c", str1[i]);

    // Copy
    for(i = 0; i <= len; i++)
        str2[i] = str1[i];

    printf("\nOriginal: %s", str1);
    printf("\nCopied: %s\n", str2);

    // Compare two strings
    char s1[100], s2[100];
    printf("Enter first string: ");
    scanf("%s", s1);
    printf("Enter second string: ");
    scanf("%s", s2);

    for(i = 0; s1[i] != '\0' || s2[i] != '\0'; i++) {
```

```

        if(s1[i] != s2[i]) {
            cmp = s1[i] - s2[i];
            break;
        }
    }

    if(cmp == 0)
        printf("Strings are equal.\n");
    else if(cmp > 0)
        printf("First string is greater.\n");
    else
        printf("Second string is greater.\n");

    // Concatenation
    i = 0;
    while(s1[i] != '\0') {
        str3[i] = s1[i];
        i++;
    }
    int j = 0;
    while(s2[j] != '\0') {
        str3[i] = s2[j];
        i++;
        j++;
    }
    str3[i] = '\0';

    printf("Concatenated String: %s\n", str3);

    return 0;
}

```